Your Data, Your Rules: Query-Layer Access for a Social Data Platform

Jacob Hoffman
web10 Inc.
jacob@web10.app

ABSTRACT

Social applications typically implement content, audience relationships, and access rules in application-specific backends. **web10** places content and access relationships in a shared data platform, allowing applications to define their own queries while users manage group membership and grants. The platform stores documents under logical user namespaces in ClickHouse and exposes generic groups, service-scoped roles, app contracts, and a query API. Its central mechanism is a query boundary: application queries name services, and the node supplies filtered relations for the reader before executing joins and aggregations. The demonstration application, web10-social, uses web10 APIs as its data and authentication backend, with social behavior implemented in a static browser frontend. It uses this mechanism to combine ranked posts, engagement, and profiles, with media capabilities prepared by the node. We describe the access model, query compilation, and application composition, including the costs of metadata resolution and the consistency obligations of a shared columnar store. The design separates application-defined social queries from shared access enforcement. This draft specifies the feasibility evaluation; latency, capacity, revocation, and cost results remain pending.

1 INTRODUCTION

For a creator, an audience is more than a list of accounts. It is a set of relationships: who follows, who belongs to a group, and who may read or contribute content. When those relationships live inside one application's backend, a new frontend must recover both the data and the rules that give it meaning. The database question is how to make content and access relationships reusable across applications while preserving the reader's authorization boundary.

web10 addresses this question with a shared data plane and a small set of composable access primitives. A user's key identifies a logical namespace, not a separate database. Generic groups associate documents with audiences; roles name per-service operations; app contracts constrain supported calling contexts. An application supplies its own services and queries rather than requiring a new feed endpoint for each ranking or audience model.

The motivating application is a deployed creator-oriented social platform, including feeds, groups, direct messages, discovery, and monetization. Its feed illustrates the separation: an application chooses chronological or weighted ranking, while the node resolves the reader's access. A chronological query makes eligible posts retrievable without an engagement-based cutoff, subject to access and moderation rules. Here distribution means eligibility for retrieval; pagination and user attention determine what is actually viewed.

This work builds on the decentralized social web. ActivityPub defines activity delivery APIs [11]; ATProto separates repositories, relays, and application views [10]; Solid separates applications from personal data stores [15]. Solid's Web Access Control supports group agents [5], and its Access Control Policy includes a client dimension [3]. web10 brings user-facing contracts and application-written SQL together within one node's shared columnar store. This placement lets an application compose cross-service reads against relations filtered for its reader, rather than define a dedicated server endpoint for each social view.

We use *generic social composition* to mean that app-defined content and relationships are represented through service-namespaced documents, generic group membership, role maps, and queries. Generic group-management endpoints are part of this substrate. The node also provides advertising, content-event, media, and result-preparation machinery. The architectural separation is between application-defined content and query semantics and these shared node services.

This paper makes three contributions:

(1) **A compositional data/access architecture** (§2): logical user namespaces, app contracts, and group roles share a service-scoped permission vocabulary, exposed to application authors and users rather than only infrastructure operators.
(2) **A reader-specific query boundary** (§3): service references resolve to filtered CTEs before caller joins and aggregation. The implementation combines policy resolution, SQL validation, and boundary construction above a shared database account.
(3) **A deployed social application case study** (§4): ranked feeds combine content, engagement, profiles, and media through application queries and node preparation. The case study shows how access scope shapes query results and follow-up reads.

The evaluation in §5 will assess practical operating points and characterize the architecture's costs. This submission's goal is feasibility and a clear account of tradeoffs; further optimization is follow-up work. Measurements remain pending in this draft. §6 relates the design to decentralized storage, IAM, and multi-tenant databases; §7 concludes.

2 THE ACCESS MODEL

2.1 Two trust layers

For a group-scoped content operation, the model distinguishes an *app*, a *principal* (the acting user), a *group* G, a *service* S (a data namespace such as `posts` or `comments`), and an *operation* O (e.g. `readAll`, `create`). Authorization asks two questions where the corresponding gates apply:

Layer	Question
App trust (outer wall)	Can this *app* perform O on S?
Person trust (inner wall)	Does membership or a class/role grant allow O on S in G?

The app contract expresses trust in a calling application. In the current document and query paths, authenticated browser-style tokens are checked against an origin-keyed contract when an HTTP `Origin` is present; without that header, this app-contract check is skipped. Anonymous queries also skip it. OAuth-delegated tokens instead select a `client_id`-keyed contract, including when no Origin is present. The node enables wildcard, non-credentialed CORS; endpoint code enforces application grants separately. Origin identifies the browser calling context, while the delegated token carries the OAuth client identity. Non-browser clients control their own Origin headers. Identity verification and document/group authorization are separate checks.

The group layer expresses trust in people. The split lets a user revoke an app grant without leaving a group, or leave a group without changing app grants.

2.2 Roles are per-service permission maps

A role is a named bundle of per-service permissions—a map from service to the operations it grants:

```
{ "name": "editor",
  "permissions": {
    "posts":    ["readAll", "create", "updateOwn"],
    "comments": ["readAll"]
  } }
```

The map *is* the scope. `'*'` wildcards all document services; a reserved key `'group'` carries *structural* operations on the group itself (`manageRoles`, `assignRoles`, `deleteGroup`, join/member operations), while *content* operations—including moderation such as `hideAll`—live under the service key, because they act on documents, not on the group's shape.

Two properties keep the model small. First, the role map is the *same shape* as the app-contract permission object: there is one permission language across both trust layers. Second, a person holds *one explicit member role* per group; compound permissions can be placed in that role rather than stacked through multiple assignments. Policies use positive grants; explicit deny and context-dependent conditions are outside this model. A group defines whatever roles fit its purpose (`owner`, `moderator`, `page-curator`, `member`, or `dj`, `listener`)— and applications supply their role definitions. Some lifecycle operations use conventional names such as `member`.

2.3 Principal classes and effective permissions

Access is granted to *principals*, of which there are three nested classes, broadest to narrowest:

Class	Who
`anyone`	every request, signed in or not
`authenticated`	a verified non-anonymous user, member or not
`member`	holds a member row in this group

The broad classes `anyone` and `authenticated` are represented by *reserved keys in the member table*; individual members use their own principal keys. No separate visibility table is needed. For example:

```
group_members:
  (G, 'anyone',        'reader')    -- the group is
       public
  (G, 'authenticated', 'reader')    -- signed-in users
       can read
  (G, 'bob',           'moderator') -- bob moderates
  (G, 'alice',         'owner')     -- alice owns
```

A principal's *effective* permissions in a group are the union of the permission maps of every class they belong to:

$$\begin{aligned}\text{effective}(P, G) \;=\; &\text{grant}(G,\ \texttt{anyone}) \\ \cup\ &\big(P \in \texttt{authenticated}\ ?\ \text{grant}(G,\ \texttt{authenticated}) : \emptyset\big) \\ \cup\ &\big(P \in \texttt{member}(G)\ ?\ \text{grant}(G,\ \text{role}(P)) : \emptyset\big)\end{aligned}$$

The content gate combines this grant union with membership admission. In `can_read_group` and `can_write_group` helpers, an active member is admitted for content reads and creates without requiring service-specific `readAll` or `create` in that member's role. For non-members, the class grants supply those permissions. Other role-controlled operations use their respective permission checks.

The union is monotone across principal classes under otherwise equal conditions. Blocks, sharing state, hidden documents, and node moderation apply additional visibility restrictions to the resulting content.

2.4 Publicness is a grant, not a flag

There is no `visibility` column. "Make my group public" is the act of granting `readAll` to `anyone`; "signed-in only" grants it to `authenticated`; "private" withholds those non-member grants. Join policy is orthogonal: it controls how a human *becomes* a member (open, request, invite-only) and says nothing about what a non-member bystander can see. A group can be invite-only to join and still publicly readable—the discovery board is exactly that.

This is the sense in which web10 brings IAM to the front: the operations a cloud operator would express as policies—*principal class, service, operation, grant*—are the operations an *end user* expresses about their own data. A grant is represented by metadata rows and revoked through tombstones. Subsequent checks use latest-row resolution. End-to-end revocation latency comprises database visibility, concurrent checks, in-flight requests, and the remaining lifetime of issued capabilities.

3 ENFORCEMENT AT THE QUERY LAYER

3.1 One columnar table

A ranked feed has analytical components: joins across authors and engagement, filtering, aggregation, and sorting. A social application also has point reads, updates, and coordination-heavy operations. web10 chooses ClickHouse [17] for both structured content and operational metadata rather than maintaining a separate Postgres-to-ClickHouse replication pipeline. User content occupies one shared `documents` table keyed by `(author_key, doc_id)`. The user namespace is logical isolation within that table, not a per-user database or physical collection. Large media objects reside in object storage. The following simplified schema omits reference and advertising fields:

```
CREATE TABLE documents (
    doc_id          String,
    author_key      String,
    collection_name String,   -- the service: 'posts', '
        comments', ...
    body            String,   -- JSON
    tags            Array(String),
    created_at      DateTime64(3),
    updated_at      DateTime64(3),
    deleted         UInt8 DEFAULT 0
) ENGINE = ReplacingMergeTree(updated_at)
ORDER BY (author_key, doc_id);
```

Updates insert a newer version; deletes insert tombstones. Background merges alone do not establish read correctness: reads must select the latest version *including tombstones*, then filter deleted rows. Filtering tombstones before deduplication can resurrect the previous version. Documents attach to groups through `doc_groups`; `group_contracts` and `group_members` hold contracts and assignments. These are narrower metadata tables whose size grows with groups and memberships.

One structured store removes a cross-database synchronization dependency. Its operational costs include version accumulation, merge pressure, read-time deduplication, point-query latency, and shared-resource contention. Multi-row content/membership changes and concurrent revocation require explicit consistency handling. Backup and recovery span database metadata and object storage; these are requirements of the design.

3.2 The query boundary: a wall, not a membrane

Application-written SQL over a multi-tenant store requires a boundary before the caller's relational operations. The query API accepts a constrained read-only SQL surface, not arbitrary database access. Its intended boundary is a *wall, not a membrane*: validated service references read reader-filtered CTEs, rather than raw tables whose output is filtered afterward. Correctness depends on validation coverage and SQL dialect interpretation, as discussed below.

Why output-side filtering is unsound. The natural design wraps the caller's query and filters the *result*, e.g. `SELECT * FROM (q) WHERE <filter>`. This fails because the caller's query has *already read the raw table* inside the parentheses, and an outer filter cannot undo an aggregation that baked everyone's data in:

```
-- counts every user's comments; the outer group filter
     is too late
SELECT author_key, count(*)
FROM documents WHERE collection_name = 'comments'
GROUP BY author_key
```

The aggregate has already collapsed all authors into counts before any row-level filter could run. Filtering the output of an aggregation is not access control.

Redirecting the input instead. web10 parses submitted SQL to an AST; it does not send that original string directly to ClickHouse. It validates table references and injects a *boundary CTE* under each referenced service name. The CTE deduplicates the service's documents, joins current `doc_groups` mappings, filters to readable groups, and applies block, sharing, hidden-document, and node-ban exclusions. Thus `SELECT ... FROM posts` resolves to a service CTE before aggregation or joins. The readable set can include public or authenticated-class grants, not just the reader's memberships. An empty set produces an empty service CTE. Before compilation, an authenticated request with explicit groups is rejected with 403 if none of its referenced services has a readable group. The query boundary does not automatically union in all reader-owned documents.

The walk and the backstop. The compilation is five steps: (1) parse to an AST (unparseable or multi-statement input is rejected); (2) check the statement kind (`SELECT` and supported set operations, not `INSERT`, `DROP`, etc.; this is not full SQL type checking); (3) walk every `Table` node, including inside caller CTEs and subqueries, rejecting raw node tables, ungranted services, and table functions (`file()`, `s3()`, `numbers()`, represented in the validated parser shapes as empty-named `Table` nodes); (4) inject the needed boundary CTEs ahead of the caller's CTEs and re-emit from the validated AST; and (5) *round-trip*—re-parse the compiled SQL and require it to come back as exactly one statement. The round-trip checks composition syntax. Targeted parser/engine tests exercise selected dialect and name-resolution cases. The executed query is emitted from the validated AST.

Why place policy above ClickHouse? web10 uses a shared database service account and constructs a reader-specific boundary combining membership, class grants, and visibility exclusions. This design puts policy orchestration in the application rather than provisioning a database account and policy for each end user. PostgreSQL policies offer another enforcement placement, using expressions, functions, and subqueries to consult dynamic authorization state [13]; an equivalent membership policy provides a comparison baseline for the cost of these enforcement placements.

Trusted implementation. The parser, validator, generated CTEs, identity resolution, metadata checks, and ClickHouse name-resolution semantics are in the trusted computing base. The service account must read raw tables to execute generated CTEs; its privileges do not independently enforce each reader's boundary. Repository tests cover selected top-level, caller-CTE, subquery, table-function, and aggregation cases, with a live-node E2E gauntlet for supported shapes. Parser and engine upgrades require renewed compatibility and adversarial testing.

3.3 Metadata resolution and relational cost

The read path is the heart of the design:

(1) **Resolve.** Take explicitly requested candidate groups, or discover the reader's memberships. Parse the query to identify its services.
(2) **Compute.** For each referenced service and candidate group, check active membership or resolve class grants through member and contract metadata. The current query endpoint calls `readable_groups`, which loops over candidates; a check may issue several metadata queries.
(3) **Execute.** Inject each service's readable-group set into its boundary CTE. ClickHouse performs document/mapping deduplication, the mapping join, visibility exclusions, and the caller's joins, aggregates, and sorting.

For s referenced services and g candidate groups, this path performs up to sg group-gate evaluations, with early membership success reducing work within a check. The repository also has a batched helper used elsewhere; the query endpoint currently uses the per-group helper. Role resolution does not need content bodies, but its metadata I/O and the generated relational work both contribute to authorization cost.

The document-to-group join can yield multiple rows for one document attached to several readable groups. Applications must choose whether to retain those associations or deduplicate before counting and pagination. CTE expansion, join cardinalities, exclusions, and sorting can change database work and query plans. Evaluation accounts for both metadata resolution and this relational work.

3.4 Isolation rule and regression coverage

The intended isolation rule is that another author's document requires an authorized group path, including public or authenticated-class grants (invariant I3). The query path implements this through service CTEs. Permission-matrix and exploit-regression tests exercise the compiler and endpoint boundaries. CRUD, directory, group-attachment, preparation, and media paths have separate test scopes; §5.4 records the current coverage and open boundaries.

3.5 Document-linked media capabilities

Structured documents refer to binary objects in MinIO/S3-compatible storage. The intended linkage is document authorization followed by capability issuance: presigned URLs let browsers fetch individual objects directly; HLS uses short-lived signed manifests and segment capabilities, with document checks on manifest access. Expiry limits capability lifetime but does not retract bytes already fetched or guarantee immediate revocation.

The query prepare pass combines media resolution, HLS signing, advertising, and avatar preparation. This keeps capability issuance next to result preparation while object storage serves the media bytes. Its authorization boundary includes key ownership, authorized cross-user references, result projections, public thumbnails, and stale capabilities (§5.4).

3.6 Data policy, not privacy

web10 deliberately trusts the node operator with readable content. Search, ranking, moderation, and audit functions operate on plaintext. Tokens and signed capabilities protect application/user boundaries; terms govern operator conduct. This trust model supports node-side querying without requiring default end-to-end encryption.

Here *user-owned* denotes logical authorship namespaces and user-controlled grants. The operator retains physical access to hosted data, and legal remedies depend on terms, jurisdiction, and enforceability. Usable export and exit are broader product goals; interoperable migration and federation remain outside the demonstrated single-node architecture.

3.7 Implementation boundary

Python/FastAPI handles request validation, identity and permission orchestration, SQLGlot parsing/compilation, serialization, and result preparation. ClickHouse's native engine performs scans, joins, aggregation, filtering, and sorting. The node API source and TypeScript SDK are available alongside the query endpoint and boundary compiler. The production API command configures four Gunicorn processes with Uvicorn workers. This service/database division of work puts query execution in ClickHouse and orchestration in Python. Profiling separates API CPU, database CPU, I/O waits, and queueing to locate the costs of parsing, metadata round trips, response serialization, and preparation.

Table 1: Media and interaction patterns covered by the demonstration. Platform names identify familiar interaction patterns.

Demonstration surface	Familiar reference
Communities, text posts and discussions	Facebook, Threads
Image and mixed-media posts	Instagram
Vertical short-video browsing	TikTok
Long-form video watch pages	YouTube
Audio-track browsing and playback	YouTube Music

4 A RUNNING SOCIAL PLATFORM

web10-social is the demonstration application built on web10: its persistent social data and authentication use web10 APIs and the SDK rather than a separate social-data backend. It exercises the architecture through feeds, social groups, direct messages, and discovery. These features require different content shapes and audience relationships over the same document and group substrate. The feed is a useful case study because it combines ranking, relationships, media, and multiple authorization scopes. We examine its query composition here; workload capacity is the subject of §5.

4.1 The killer-app hypothesis

web10-social is also the project's attempt at a *killer app*: an application compelling enough to attract users to a new platform. The longer-term vision is an open, decentralized ecosystem in which many applications reuse a shared data and access substrate. Social is the initial adoption vehicle and a demanding demonstration of that substrate. Adoption remains an open product goal; the case study examines the implemented application breadth.

The application spans a media spectrum from text and social relationships to images, short-form video, long-form video, and audio. Facebook, Instagram, Threads, TikTok, YouTube, and YouTube Music provide familiar reference points for these interaction patterns. Table 1 maps them to the demonstration's implemented surfaces. These reference points describe interaction patterns; the case study evaluates their composition on web10.

Different media and interaction patterns share document storage, audience relationships, engagement records, and access machinery, while their presentation and queries are defined by the client. The architectural demonstration is that this range of social experiences can be built on web10's APIs. A successful adoption vehicle would bring those users and their relationships into the wider application ecosystem.

4.2 A frontend application over the shared node

The application is a React/TypeScript SPA. Its data client connects to the node; social operations express posts, conversations, relationships, and feeds through web10's document, group, query, and media interfaces. This is backendless from the application developer's perspective: web10 supplies the backend, while the application supplies its data conventions, queries, and interface. The browser build consists of static assets and can be hosted independently, for example in S3 behind HTTPS/CDN and SPA route handling.

The current deployment also includes a small Node link-preview renderer for crawlers. That presentation component uses web10 APIs and holds no separate social-data store; a static-only deployment needs a separate arrangement for crawler cards. Media bytes transfer through API-issued object-storage URLs, and real-time peer traffic uses the SDK's RTC support. Telemetry and external embeds are additional network paths. Application data and authentication use the node; media delivery and these ancillary connections follow their respective transport paths.

At source revision `317fa349e`, the application-local frontend contains **46,285 physical source lines across 141 files**: 45,846 TypeScript/TSX lines and 439 CSS lines under `src/`, excluding tests. Its local tests contain another 31,060 lines in 105 files; the link-preview renderer is 431 lines in three JavaScript modules. Counts include comments, blank lines, and local type declarations. They exclude public/generated assets, dependencies, configuration, tooling, and sibling shared-library and SDK implementations. All implementation links use the same pinned revision.

4.3 The feed as a query

The application's feed implementation builds a query over `posts`, `reactions`, `comments`, and `profile`. It chooses chronological or weighted ranking, joins engagement aggregates and author profiles, and supplies cursor conditions. The node resolves those service names through the boundary CTEs. Preparation can attach advertising and resolve media, HLS manifests, and author avatars on returned rows. This separates the application's feed definition from a dedicated feed endpoint, while retaining explicit server-side preparation machinery.

Three implementation observations define the scope of the case study:

(1) **Page assembly.** For a nonempty page with post IDs, `readFeedPage` issues the main query, then two engagement queries (reactions and comments) and a `contentViews` call. Group discovery precedes this work, and media bytes require separate fetches. UI-level reaction-state reads and other screen activity may add calls. Page assembly therefore combines one main composition query with scoped follow-up reads.

(2) **Authorization scope affects semantics.** The main query's engagement joins use the feed-group scope, while engagement documents may live in the discovery group. The follow-up queries use the union of feed groups and discovery to populate displayed counts. Counts are relative to readable relations, not necessarily global totals. The displayed counts can therefore differ from the counts used by the main ranking query.

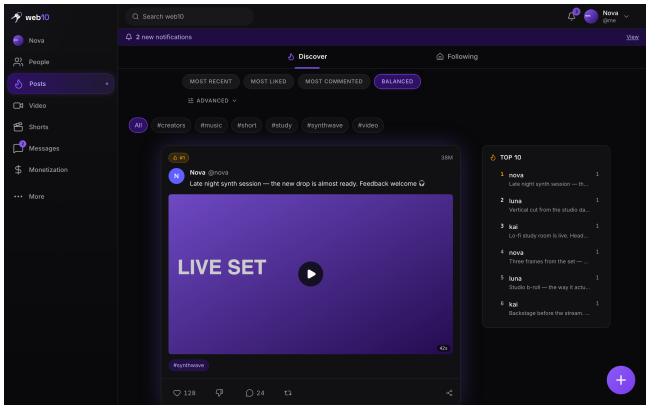


Figure 1: The web10 feed. The reader chooses the ranking (Most Recent / Most Liked / Most Commented / Balanced) and filters (tags); each is a different application query over the same columnar store. The main query uses reader-specific service boundaries; engagement and view metrics also use follow-up calls.

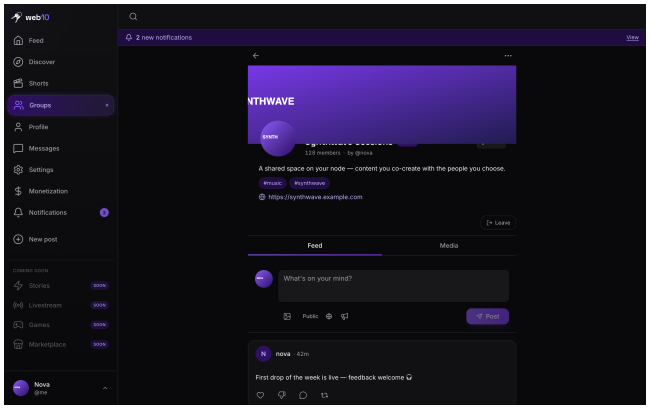


Figure 2: A group—"a shared space on your node." Membership, roles, and per-post visibility (Public / node / group) expose application choices over the generic groups and grants of §2.

(3) **The query is application-defined.** Ranking and filtering can change without adding a dedicated node feed endpoint. Applications can select their ranking and projections while reusing the same boundary construction and preparation services.

Evaluation will measure both the complete page-assembly path and its main query, recording client-call counts, internal metadata queries, and database work separately alongside latency and throughput.

4.4 Durability and extensibility

Application-named services let different clients interpret documents without changing the shared document schema. A conversation can be a group with message documents; a music library can define track services and listener roles; a shop can define catalog services.

Domain-specific transactions, payment workflows, search indexes, schema evolution, and external integrations can extend this common substrate.

The durable separation is between stored content/access relationships and a particular frontend. Replacing a frontend can reuse those primitives when it understands their semantics. Moving providers is a larger task: identity continuity, document references, group grants, media, and remote relationships must survive export/import. The implemented query boundary is single-node; provider migration and federation are future work.

5 EVALUATION STATUS AND METHODOLOGY

5.1 Evidence available and pending

The current case study documents the implemented architecture, repository tests for named paths, and a deployed application. Empirical evaluation is pending. The planned measurements assess workload-specific feasibility and costs: production observations, latency distributions, throughput, resource use, revocation visibility, and matched baseline behavior.

The DigitalOcean experiment scaffold includes provisioning, collection scripts, and a documented blocker list; the evaluation plan defines the required workload and measurement checks. Existing social gauntlets provide reusable correctness scenarios for following, posting, and concurrent engagement. These artifacts support construction of the connected application workload; execution and result collection remain pending.

An industry evaluation needs dated production observations separated from synthetic runs. Production observations should identify deployment duration, registered and active users, live and versioned document counts, services, groups, membership distributions, object volume, and operation rates. Logical user namespaces are counted separately from physical storage. Availability, recovery, and operating costs use explicit observation windows, failure definitions, and accounting boundaries.

5.2 Workloads and measurement

Experiments should pin the source revision, dependency and engine versions, schema, data generator and seed, hardware, storage, network placement, worker counts, connection limits, and database settings. A localhost harness excludes wide-area latency but still includes scheduling, queueing, container networking, and service waits; it is not a browser experience measurement. Browser traces must separately capture group discovery, main and follow-up queries, media fetches, and time to a defined render milestone.

Vary document count, versions per document, groups per reader, services per query, group overlap, response size, ranking complexity, and concurrent writes. Include member, public-class, authenticated-class, denied, and anonymous reads, as well as posting, direct-message, and group flows. Multi-group attachments need explicit duplicate/-counting semantics. State warm-up, cache conditions, run length, offered-load model, concurrency, repeated runs, and uncertainty. Report successful throughput, errors and timeouts alongside end-to-end p50/p95/p99 latency; a latency distribution of successful requests alone can hide overload. Separate API and ClickHouse CPU/memory, storage growth, merge activity, internal query counts, and resource cost.

The query telemetry times authentication, app contracts, parsing/validation, candidate discovery, readable-group resolution, group metadata, compilation, execution, serialization, preparation, and total handler duration. Phase wall times include waits and do not directly measure Python CPU. The ratio of the median readable-group phase to median execution time is a descriptive ratio, not total authorization overhead: policy also changes compilation and the SQL executed, and ratios of medians are not distributions of per-request overhead. Instrumentation coverage, including work outside named phases, must be stated.

5.3 Baselines and correctness

A conventional shared-table backend with dynamic PostgreSQL RLS provides a baseline [13]. Match returned content, membership/class grants, visibility exclusions, ranking, pagination, preparation, consistency requirements, and hardware/resource budgets. Document indexes, caching, batching, and implementation differences. Compare full flows as well as database queries. Both systems use shared tables and logical user namespaces.

A separate authorization-disabled ablation on isolated synthetic data can measure the cost of checks removed from an otherwise matched request. Account for both metadata orchestration and changes to generated query plans; retain equivalent access semantics for the product baseline. Comparisons with Pod or federated architectures also account for placement, autonomy, caching, and network topology.

Correctness evaluation should publish a pinned test matrix and actual results, distinguishing unit tests, mocked endpoints, and live-store and browser tests. Cover principal classes, membership admission, service permissions, tombstones, multi-group rows, aggregations, nested SQL, denied raw tables/functions, and capability issuance. Revocation experiments must race grant changes with reads and measure stale access through metadata visibility, in-flight requests, and issued URLs/manifests.

5.4 Scope and open boundaries

This case study covers a single-node architecture and implemented application patterns. Production-scale adoption, exhaustive competitor feature parity, catalog and recommendation quality, and cross-node migration are outside the evaluation. Performance results will be reported for the measured workloads and configurations.

Security evidence consists of regression tests for named paths. Whole-system conformance and the ongoing credential, consent/delegation, and media reviews remain open. In particular, a separate read-URL handler forwards caller-selected keys to signing without an intervening object check. The recorded local probe demonstrates that forwarding; actual private-object retrieval remains unverified. Parser/engine semantic differences and result-projection provenance also require targeted testing. These boundaries are part of the follow-up correctness work alongside performance optimization.

6 RELATED WORK

Decentralized social data. ActivityPub [11] defines client/server and federated server/server activity delivery. ATProto's repositories, relays, and AppViews separate storage from application views and support user agency and provider switching [10]. web10 focuses on query composition within a node: app-defined service queries read a shared store through relations filtered for the reader at request time. Federation and provider migration address complementary system boundaries.

Solid separates applications from personal data stores [15], with social application work [14] and a maintained Community Solid Server [7]. WAC supports resource permissions and group agents through `acl:agentGroup` [5]; ACP includes agent, client, and issuer matching [3]. web10 packages group membership, join policies, and per-service role maps with a shared relational query path. Pods are logical stores that need not each occupy a separate server; cross-store query cost depends on deployment, caching, indexing, and query strategy. The comparison here concerns data placement and query interfaces; a performance comparison would require a matched workload and deployment.

Cloud IAM. AWS IAM (documentation) [1] and the RBAC/ABAC literature [6, 8, 16] provide the policy concepts web10 adapts at the user boundary: principals, roles, service-scoped operations, revocable grants. The move in this paper is directional—a familiar policy vocabulary exposed to users about their own content and groups, orchestrated by the API and query layer. At industrial scale, relationship-based authorization (Zanzibar [12]) shows a uniform ACL data model and consistency as a first-class problem. Its consistency guarantees motivate explicit evaluation of web10's revocation visibility. TAO [4] provides a social-graph storage precedent, whereas web10 composes application queries over service-namespaced documents in a columnar store.

Multi-tenant isolation. The database literature on tenant isolation—shared tables with an organization identifier (Force.com [18]), schema-mapping techniques [2], and the shared-versus-separate spectrum [9]—is the closest systems ancestor. web10's key-space per user makes the *end user* the logical tenant. Authorship uses `author_key`; shared reads use authorized group relations rather than a physical database boundary. PostgreSQL row-security policies [13] are an alternative location for dynamic access enforcement. Matched policy and workload execution (§5) can distinguish the costs of these enforcement placements. ClickHouse [17] supplies the columnar execution engine; web10 supplies the access orchestration above it, including metadata resolution, SQL compilation, and result preparation.

7 CONCLUSION

web10 makes content and access relationships a shared substrate for application queries. Logical user namespaces, app contracts, and generic groups connect user-managed access to a columnar store; the query layer supplies the reader's service relations before joins and aggregation. The social application shows how this separation supports ranked feeds and scoped engagement reads while keeping query definitions in the application.

The next step is to establish a practical operating point and characterize metadata fan-out, query execution, and revocation under representative workloads. Those measurements remain pending. They will provide a baseline for subsequent optimization of policy resolution and query processing. The architectural contribution is this integration: applications compose social views over reader-specific service relations in a shared columnar store.

REFERENCES

[1] Amazon Web Services. 2025. AWS Identity and Access Management (IAM) User Guide. Amazon Web Services documentation. https://docs.aws.amazon.com/IAM/latest/UserGuide/introduction.html

[2] Stefan Aulbach, Torsten Grust, Dean Jacobs, Alfons Kemper, and Jan Rittinger. 2008. Multi-Tenant Databases for Software as a Service: Schema-Mapping Techniques. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. ACM, 1195–1206.

[3] Matthieu Bosquet. 2022. Access Control Policy (ACP). W3C Solid Community Group, Editor's Draft. https://solid.github.io/authorization-panel/acp-specification/

[4] Nathan Bronson, Zach Amsden, George Cabrera, Prasad Chakka, Peter Dimov, Hui Ding, Jack Ferris, Anthony Giardullo, Sachin Kulkarni, Harry C. Li, Mark Marchukov, Dmitri Petrov, Lovro Puzar, Yee Jiun Song, and Venkateshwaran Venkataramani. 2013. TAO: Facebook's Distributed Data Store for the Social Graph. In *Proceedings of the 2013 USENIX Annual Technical Conference (USENIX ATC '13)*. USENIX Association, 49–60.

[5] Sarven Capadisli. 2022. Web Access Control. W3C Solid Community Group, Version 1.0.0-cr-1. https://solidproject.org/TR/wac

[6] David F. Ferraiolo and David R. Kuhn. 1992. Role-Based Access Controls. In *Proceedings of the 15th National Computer Security Conference*. 554–563.

[7] Joachim Van Herwegen and Ruben Verborgh. 2024. The Community Solid Server: Supporting Research and Development in an Evolving Ecosystem. *Semantic Web* 15, 6 (2024), 2597–2611. https://doi.org/10.3233/SW-243726

[8] Vincent C. Hu, David F. Ferraiolo, David R. Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. 2014. *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Technical Report SP 800-162. National Institute of Standards and Technology. Includes updates as of August 2019.

[9] Dean Jacobs and Stefan Aulbach. 2007. Ruminations on Multi-Tenant Databases. In *Proceedings of BTW 2007*. 514–521.

[10] Martin Kleppmann, Paul Frazee, Jake Gold, Jay Graber, Daniel Holmgren, Devin Ivy, Jeromy Johnson, Bryan Newbold, and Jaz Volpert. 2024. Bluesky and the AT Protocol: Usable Decentralized Social Media. In *Proceedings of the ACM CoNEXT-2024 Workshop on the Decentralization of the Internet (DIN '24)*. ACM, 9. https://doi.org/10.1145/3694809.3700740 Extended version: arXiv:2402.03239.

[11] Christine Lemmer-Webber, Jessica Tallon, Erin Shepherd, Amy Guy, and Evan Prodromou. 2018. ActivityPub. W3C Recommendation. https://www.w3.org/TR/activitypub/ Editors: Christine Lemmer-Webber and Jessica Tallon.

[12] Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. 2019. Zanzibar: Google's Consistent, Global Authorization System. In *Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)*. USENIX Association. https://www.usenix.org/system/files/atc19-pang.pdf

[13] PostgreSQL Global Development Group. n.d. PostgreSQL Documentation: Row Security Policies. PostgreSQL documentation. https://www.postgresql.org/docs/current/ddl-rowsecurity.html Accessed October 2026.

[14] Andrei Sambra, Amy Guy, Sarven Capadisli, and Nicola Greco. 2016. Building Decentralized Applications for the Social Web. In *Proceedings of the 25th International Conference Companion on World Wide Web (WWW '16 Companion)*. ACM, 1033–1034.

[15] Andrei Vlad Sambra, Essam Mansour, Sandro Hawke, Maged Zereba, Nicola Greco, Abdurrahman Ghanem, Dmitri Zagidulin, Ashraf Aboulnaga, and Tim Berners-Lee. 2016. *Solid: A Platform for Decentralized Social Applications Based on Linked Data*. Technical Report. MIT CSAIL and Qatar Computing Research Institute. http://emansour.com/research/lusail/solid_protocols.pdf

[16] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. 1996. Role-Based Access Control Models. *IEEE Computer* 29, 2 (1996), 38–47.

[17] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse: Lightning Fast Analytics for Everyone. *Proceedings of the VLDB Endowment* 17, 12 (2024), 3731–3744. https://doi.org/10.14778/3685800.3685802

[18] Craig D. Weissman and Steve Bobrowski. 2009. The Design of the Force.com Multitenant Internet Application Development Platform. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data (SIGMOD '09)*. ACM, 889–896. https://doi.org/10.1145/1559845.1559942